Week 3: Data Preparation, Visualization and Feature Extraction

Matplotlib

Matplotlib is an amazing visualization library in Python for 2D plots of arrays. Matplotlib is a multi-platform data visualization library built on NumPy arrays and designed to work with the broader SciPy stack. It was introduced by John Hunter in the year 2002. One of the greatest benefits of visualization is that it allows us visual access to huge amounts of data in easily digestible visuals. Matplotlib consists of several plots like line, bar, scatter, histogram etc.

Basic plots in Matplotlib :

Matplotlib utilities lies under the pyplot submodule, and are usually imported under the plt alias: import

matplotlib.pyplot as plt or from matplotlib import pyplot as plt

The plot() function is used to draw points (markers) in a diagram. By default, the plot() function draws a line from point to point. The function takes parameters for specifying points in the diagram. Parameter 1 is an array containing the points on the x-axis. Parameter 2 is an array containing the points on the y-axis.

The plt.show() command interacts with our system's interactive graphical backend. plt.show() command should be used only once per Python session, and is most often seen at the very end of the script. Multiple show() commands can lead to unpredictable backend-dependent behavior, and should mostly be avoided.

1. Line plot :

```
import matplotlib.pyplot as plt
 x = [5, 2, 9, 4, 7]
# Y-axis values
y = [10, 5, 8, 4, 2]
# Function to plot
plt.plot(x,y)
# function to show the plot
plt.show()
```

2. Bar plot:

With pyplot, we can use the bar() function to draw bar graphs. The bar() function takes arguments that describes the layout of the bars. The categories and their values are represented by the first and second argument as arrays.

```
import matplotlib.pyplot as plt
x = [5, 2, 9, 4, 7]
# Y-axis values
y = [10, 5, 8, 4, 2]
# Function to plot the bar
plt.bar(x,y)
```

3. Hist plot:

A histogram is basically used to represent data provided in a form of some groups. It is accurate method for the graphical representation of numerical data distribution. It is a type of bar plot where X-axis represents the bin ranges while Y-axis gives information about frequency.

Creating a Histogram

To create a histogram the first step is to create bin of the ranges, then distribute the whole range of the values into a series of intervals, and count the values which fall into each of the intervals. Bins are clearly identified as consecutive, non-overlapping intervals of variables.

The matplotlib.pyplot.hist() function is used to compute and create histogram of x. In Matplotlib, we use the hist() function to create histograms. The hist() function will use an array of numbers to create a histogram, the array is sent into the function as an argument.
Example: Say you ask for the height of 250 people, you might end up with a histogram like this: The

hist() function will read the array and produce a histogram:

You can read from the histogram that there are approximately: 2
people from 140 to 145cm
5 people from 145 to 150cm
15 people from 151 to 156cm
31 people from 157 to 162cm
46 people from 163 to 168cm
53 people from 168 to 173cm
45 people from 173 to 178cm
28 people from 179 to 184cm
21 people from 185 to 190cm
4 people from 190 to 195cm

Simple histogram Template:

```
import matplotlib.pyplot as plt

x = [value1, value2, value3,..... ]

plt.hist(x, bins=number of bins)

plt.show()
```

Attribute parameter

x : array or sequence of array

bins : optional parameter contains

integer or sequence or string

```
Age

1, 1, 2, 3, 3, 5, 7, 8, 9, 10,
10, 11, 11, 13, 13, 15, 16, 17, 18, 18,
18, 19, 20, 21, 21, 23, 24, 24, 25, 25,
25, 25, 26, 26, 26, 27, 27, 27, 27, 27,
29, 30, 30, 31, 33, 34, 34, 34, 35, 36,
36, 37, 37, 38, 38, 39, 40, 41, 41, 42,
43, 44, 45, 45, 46, 47, 48, 48, 49, 50,
51, 52, 53, 54, 55, 55, 56, 57, 58, 60,
61, 63, 64, 65, 66, 68, 70, 71, 72, 74,
75, 77, 81, 83, 84, 87, 89, 90, 90, 91
```

Step 2: Determine the number of bins

Next, determine the number of bins to be used for the histogram. For simplicity, set the number of bins to

- At the end we will see another

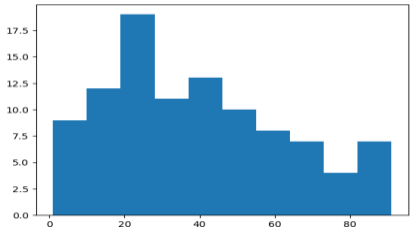
  way to derive the bins.

Step 3: Plot the histogram in Python

using

matplotlib import matplotlib.pyplot as plt

```
x = [1, 1, 2, 3, 3, 5, 7, 8, 9, 10,
  10, 11, 11, 13, 13, 15, 16, 17, 18, 18,
  18, 19, 20, 21, 21, 23, 24, 24, 25, 25,
  25, 25, 26, 26, 26, 27, 27, 27, 27, 27,
  29, 30, 30, 31, 33, 34, 34, 34, 35, 36,
  36, 37, 37, 38, 38, 39, 40, 41, 41, 42,
  43, 44, 45, 45, 46, 47, 48, 48, 49, 50,
  51, 52, 53, 54, 55, 55, 56, 57, 58, 60,
  61, 63, 64, 65, 66, 68, 70, 71, 72, 74,
  75, 77, 81, 83, 84, 87, 89, 90, 90, 91]
plt.hist(x, bins=10)
plt.show()
```



Additional way to determine the number of bins

Originally, we set the number of bins to 10 for simplicity.Alternatively, we may derive the bins using the following formulas:

n = number of observations

Range = maximum value – minimum value

Number of intervals = $\sqrt{n}$

Width of intervals = Range / (Number of intervals)

Age

1, 1, 2, 3, 3, 5, 7, 8, 9, 10,
10, 11, 11, 13, 13, 15, 16, 17, 18, 18,
18, 19, 20, 21, 21, 23, 24, 24, 25, 25,
25, 25, 26, 26, 26, 27, 27, 27, 27, 27,
29, 30, 30, 31, 33, 34, 34, 34, 35, 36,
36, 37, 37, 38, 38, 39, 40, 41, 41, 42,
43, 44, 45, 45, 46, 47, 48, 48, 49, 50,
51, 52, 53, 54, 55, 55, 56, 57, 58, 60,
61, 63, 64, 65, 66, 68, 70, 71, 72, 74,
75, 77, 81, 83, 84, 87, 89, 90, 90, 91

These formulas can then be used to create the frequency table followed by the histogram. Recall

that our dataset contained the following 100 observations:

n = number of observations = 100

Range = maximum value – minimum value = 91 – 1 = 90 Number

of intervals = √n = √100 = 10

Width of intervals = Range / (Number of intervals) = 90/10 = 9 Based on

this information, the frequency table would look like this:

Intervals (bins)	Frequency
0-9	9
10-19	13
20-29	19
30-39	15
40-49	13
50-59	10
60-69	7
70-79	6
80-89	5
90–99	3

Note that the starting point for the first interval is 0, which is very close to the minimum observation of 1 in our dataset. If, for example, the minimum observation was 20, then the starting point for the first interval should be 20, rather than 0.
For the *bins* in the Python code below, we need to specify the values highlighted in blue, rather than a particular number (such as 10, which we used before). Also, we need to include the last value of 99.
This is how the Python code would look like:

```
import matplotlib.pyplot as plt

x = [1, 1, 2, 3, 3, 5, 7, 8, 9, 10,
  10, 11, 11, 13, 13, 15, 16, 17, 18, 18,
  18, 19, 20, 21, 21, 23, 24, 24, 25, 25,
  25, 25, 26, 26, 26, 27, 27, 27, 27, 27,
  29, 30, 30, 31, 33, 34, 34, 34, 35, 36,
  36, 37, 37, 38, 38, 39, 40, 41, 41, 42,
  43, 44, 45, 45, 46, 47, 48, 48, 49, 50,
  51, 52, 53, 54, 55, 55, 56, 57, 58, 60,
  61, 63, 64, 65, 66, 68, 70, 71, 72, 74,
  75, 77, 81, 83, 84, 87, 89, 90, 90, 91
  ]
plt.hist(x, bins=[0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 99])
plt.show()
```

Sactter Plot

A scatter plot is a diagram where each value in the data set is represented by a dot. The Matplotlib module has a method for drawing scatter plots, it needs two arrays of the same length, one for the values of the x-axis, and one for the values of the y-axis.

Example

```
# importing matplotlib module  from
matplotlib import pyplot as plt # x-axis
values
x = [5, 2, 9, 4, 7]
# Y-axis values
```

```
y = [10, 5, 8, 4, 2]
# Function to plot scatter plt.scatter(x,
y)
# function to show the plot plt.show()
```

SciPy:

SciPy is a scientific computation library that uses NumPy underneath. SciPy stands for Scientific Python. It provides more utility functions for optimization, stats and signal processing. Like NumPy, SciPy is open source so we can use it freely. SciPy was created by NumPy's creator Travis Olliphant. SciPy has optimized and added functions that are frequently used in NumPy and Data Science. SciPy is predominantly written in Python, but a few segments are written in C. Once SciPy is installed, import the SciPy module(s) you want to use in your applications by adding the from scipy import module statement:

```
from scipy import constants
```

constants: SciPy offers a set of mathematical constants, one of them is liter which returns 1 liter as cubic meters.

Unit Categories: Metric, Binary, Mass, Angle, Time, Length, Pressure, Volume, Speed, Temperature, Energy, Power, Force

Example 6

```
from scipy import constants print(constants.peta
                                 #1000000000000000.0
print(constants.tera)          #1000000000000.0
print(constants.giga)           #1000000000.0
print(constants.mega)            #1000000.0
print(constants.kilo)          #1000.0
print(constants.hecto)          #100.0
print(constants.deka)           #10.0
print(constants.deci)          #0.1
print(constants.centi)          #0.01
print(constants.milli)          #0.001
print(constants.micro)           #1e-06
print(constants.nano)           #1e-09
print(constants.pico)          #1e-12
```

Sparse Data:

Sparse data is data that has mostly unused elements (elements that don't carry any information). It can be an array like this one: [1, 0, 2, 0, 0, 3, 0, 0, 0, 0, 0, 0]**.** It is a data set where most of the item values are zero. **Dense Array:** is the opposite of a sparse array: most of the values are *not* zero. In scientific computing, when we are dealing with partial derivatives in linear algebra we will come across sparse data.bSciPy has a module, scipy.sparse that provides functions to deal with sparse data. There are primarily two types of sparse matrices that we use:

- CSC - Compressed Sparse Column. For efficient arithmetic, fast column slicing.
- CSR - Compressed Sparse Row. For fast row slicing, faster matrix vector products We will use the CSR matrix.

CSR Matrix

We can create CSR matrix by passing an arrray into function scipy.sparse.csr_matrix().

Example

Create a CSR matrix from an array:

```
import numpy as np
from scipy.sparse import csr_matrix arr =
np.array([0, 0, 0, 0, 0, 1, 1, 0, 2])
print(csr_matrix(arr))
```

Sparse Matrix Methods

Viewing stored data (not the zero items) with the data property:

Example

```
import numpy as np
from scipy.sparse import csr_matrix
arr = np.array([[0, 0, 0], [0, 0, 1], [1, 0, 2]])
print(csr_mat

rix(arr).data)

[1 1 2]
```

Converting from csr to csc with the tocsc() method:

Example

```
import numpy as np
from scipy.sparse import csr_matrix
arr = np.array([[0, 0, 0], [0, 0, 1], [1, 0, 2]])
newarr =
csr_matrix(arr).tocsc()
print(newarr)
```

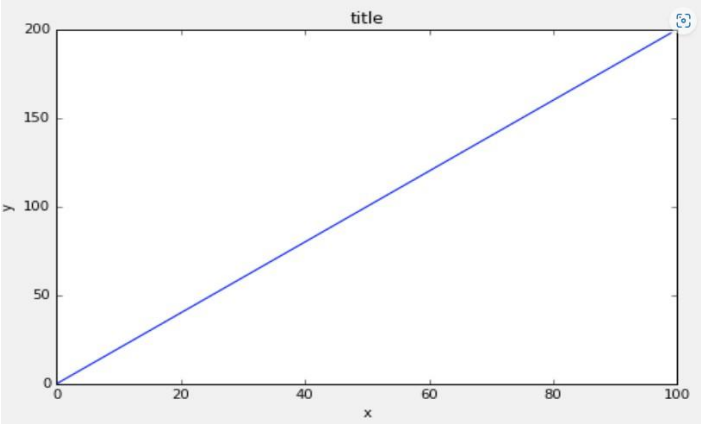
Questions

1. Follow along with these steps:

a) Create a figure object called fig using plt.figure()

b) Use add_axes to add an axis to the figure canvas at [0,0,1,1]. Call this new axis ax.

c) Plot (x,y) on that axes and set the labels and titles to match the plot below:



2. Create a figure object and put two axes on it, ax1 and ax2. Located at [0,0,1,1] and [0.2,0.5,.2,.2] respectively. Now plot (x,y) on both axes. And call your figure object to show it.

3. Use the company sales dataset csv file, read Total profit of all months and show it using a line plot.

4. A retail company recorded advertising_spend and monthly_sales for 50 stores. Plot a scatter plot to visualize the relationship. Based on the pattern, would you recommend increasing the ad budget? Support your answer with the correlation coefficient.

5. A school wants to know if hours_studied predicts exam_score for 200 students, but also wants to see if this relationship differs between students who attended extra tutoring vs. those who didn't. Create a scatter plot color-coded by tutoring status and interpret the result.

6. A hospital tracked patient_wait_time (in minutes) across 500 visits. Plot a histogram to assess whether wait times are consistent or highly variable. Identify any skewness and suggest what might be causing it (e.g., emergency cases delaying routine ones).

7. A professor wants to check whether an exam was "fair" by plotting a histogram of exam_scores for 150 students. Determine whether the distribution looks normal, and discuss whether the professor should curve the grades based on the shape.

8. An online store recorded delivery_delay_days for two warehouses, A and B. Overlay their histograms with transparency to compare which warehouse has more consistent (lower variance) delivery performance.

9. An e-commerce platform has a user_item_matrix where rows are users, columns are products, and values are ratings (1–5), but 95% of the entries are zero (unrated). Explain why storing this matrix as CSR is more practical than as a dense NumPy array, and convert a sample 6×6 ratings matrix to CSR format.

10. A social media platform models "friend connections" between 100,000 users as an adjacency matrix, where most users are connected to only a handful of others. Store this network as a CSR matrix and perform a matrix-vector multiplication to compute the number of friends each user has (row sum).

11. A gene-expression dataset has 20,000 genes (rows) and 1,000 samples (columns), where only a small number of genes are actively expressed in each sample. Convert this dataset to CSR format, then extract and visualize (using a histogram) the number of non-zero gene expressions per sample.

12. You're given a sparse user_movie_rating matrix (CSR format) from a streaming platform.
    a) Compute the number of ratings per user and plot a histogram of this distribution.
    b) Pick two popular movies and create a scatter plot of their ratings across users who rated both (after converting relevant columns to dense).
    c) Based on both visualizations, write a short summary of user rating behavior (e.g., are most users rating only a few movies? Do the two movies show correlated ratings, suggesting similar audiences?).